

浙江大学实验报告

姓名：李彦萱 专业：电子科学与技术 学号：3250100761

课程名称：数字系统 任课老师：李宇波

实验名称：Lab5 数码管滚动显示字符串 实验日期：2026.4.11

1 实验目的和要求

1.1 实验目的

如何利用有限状态机（FSM）实现字符串循环显示功能。

1.2 实验要求

（1）设计一个有限状态机（FSM）电路，用于以 0.5Hz 的速率循环显示字符串（当前为 20260411）的每个数字位，并在字符串空格处使显示屏熄灭（即关闭所有 LED 灯段）。假设系统主时钟频率为 100MHz，并编写该设计的 Verilog 代码。

（2）在 Basys3 开发板上实现上述电路。

（3）以 0.5Hz 的频率在板上循环显示上述字符串。

2 实验原理

利用状态机实现七段数码管显示状态的变化。

3 实验内容

（1）以 0.5Hz 的速率循环显示字符串的每个数字位，并在字符串空格处使显示屏熄灭

根据分析，在这一步的实验中我需要在数码管上以接续的方式显示 2026 04 11，总共有 10 个状态。然后对于 0.5Hz 这一要求，我在编写程序时需要引入分频器。

以下为全部代码：

分频器：

```
module clk_divider(  
    input wire clk,  
    input wire reset,  
    output wire tick_2s  
);  
    reg [27:0] count;  
    always @(posedge clk or posedge reset) begin  
        if (reset)  
            count <= 28'd0;  
        else if (count == 28'd199_999_999)  
            count <= 28'd0;  
        else  
            count <= count + 1;  
        end  
        assign tick_2s = (count == 28'd199_999_999);  
endmodule
```

状态机：

```
module char_fsm(  
    input wire clk,  
    input wire reset,  
    input wire tick_2s,
```

```

    output reg [6:0] seg // [g f e d c b a]
);
reg [3:0] state;

always @(posedge clk or posedge reset) begin
    if (reset)
        state <= 4'd0;
    else if (tick_2s) begin
        if (state == 4'd9)
            state <= 4'd0;
        else
            state <= state + 1;
    end
end

always @(*) begin
    case(state)
        4'd0: seg = 7'b1011011;
        4'd1: seg = 7'b0111111;
        4'd2: seg = 7'b1011011;
        4'd3: seg = 7'b1111101;
        4'd4: seg = 7'b0000000;
        4'd5: seg = 7'b0111111;
        4'd6: seg = 7'b1100110;
        4'd7: seg = 7'b0000000;
        4'd8: seg = 7'b0000110;
        4'd9: seg = 7'b0000110;
        default: seg = 7'b0000000;
    endcase
end
Endmodule

```

顶层模块:

```

module top_lab5(
    input wire clk,
    input wire reset_btn,
    output wire [6:0] seg,
    output wire [3:0] an
);
    wire tick;
    clk_divider u_clk_div (
        .clk(clk),
        .reset(reset_btn),
        .tick_2s(tick)
    );

    char_fsm u_fsm (
        .clk(clk),
        .reset(reset_btn),

```

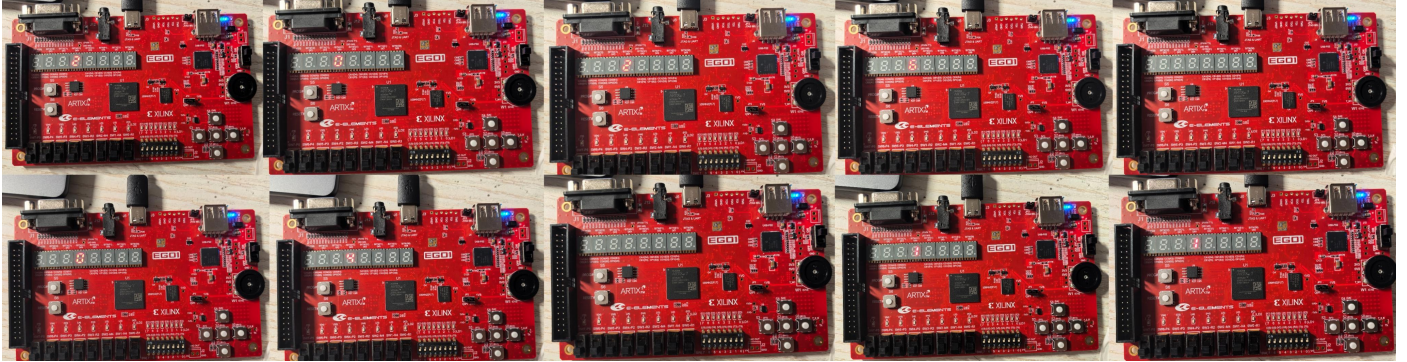
```

        .tick_2s(tick),
        .seg(seg)
    );
    assign an = 4'b1000;

```

Endmodule

运行结果如下：



(2) 以 0.5 赫兹的频率将上述字符串以四个 7 段 LED 灯管的循环方式显示

感觉在基础原理上和第一部分差别不是很大，还是使用分频器来约束 5HZ 的条件，通过用八个数码管在 clk 反转时同步切换状态来实现，总状态数变为八个。此外因为一组数码管只有一个段选信号输入，无法实现同时亮起不同的数字。所以还要考虑时分复用，即根据人眼的视觉残留效应。如果我们切换数码管的速度足够快，在人看来，这 8 位数码管就像是同时亮着的。在滚动的实现上需要通过状态的转变来实现。所以为了同时实现扫描和滚动，分频器需要产生两个不同频率的使能脉冲。

代码如下：

顶层模块：

```

module top_lab5(
    input wire clk,
    input wire reset_btn,
    output wire [6:0] seg_L,
    output wire [6:0] seg_R,
    output wire [7:0] an
);
    wire t1ms, t2s;
    wire [6:0] pattern;

    clk_divider u_div (
        .clk(clk),
        .reset(reset_btn),
        .tick_1ms(t1ms),
        .tick_2s(t2s)
    );

    display_ctrl u_ctrl (
        .clk(clk),
        .reset(reset_btn),
        .tick_1ms(t1ms),
        .tick_2s(t2s),
        .an_out(an),
        .seg_pattern(pattern)
    );

```

```

);
assign seg_L = pattern;
assign seg_R = pattern;
Endmodule

```

分频器:

```

module clk_divider(
    input wire clk,
    input wire reset,
    output wire tick_1ms,
    output wire tick_2s
);
    reg [16:0] cnt_scan;
    reg [27:0] cnt_move;

    always @(posedge clk or posedge reset) begin
        if (reset) cnt_scan <= 0;
        else if (cnt_scan == 17'd99_999) cnt_scan <= 0;
        else cnt_scan <= cnt_scan + 1;
    end
    assign tick_1ms = (cnt_scan == 17'd99_999);

    always @(posedge clk or posedge reset) begin
        if (reset) cnt_move <= 0;
        else if (cnt_move == 28'd199_999_999) cnt_move <= 0;
        else cnt_move <= cnt_move + 1;
    end
    assign tick_2s = (cnt_move == 28'd199_999_999);
Endmodule

```

控制滚动:

```

module display_ctrl(
    input wire clk,
    input wire reset,
    input wire tick_1ms,
    input wire tick_2s,
    output reg [7:0] an_out,
    output reg [6:0] seg_pattern
);

    reg [31:0] data_reg;
    reg [2:0] scan_cnt;
    reg [3:0] hex_val;

    always @(posedge clk or posedge reset) begin
        if (reset)
            data_reg <= {4'd2, 4'd0, 4'd2, 4'd6, 4'd0, 4'd4, 4'd1, 4'd1};
        else if (tick_2s)
            data_reg <= {data_reg[27:0], data_reg[31:28]};
    end

```

```

end

always @(posedge clk or posedge reset) begin
    if (reset) scan_cnt <= 0;
    else if (tick_1ms) scan_cnt <= scan_cnt + 1;
end

always @(*) begin
    an_out = 8'b0000_0000;
    an_out[scan_cnt] = 1'b1;

    case(scan_cnt)
        3'd0: hex_val = data_reg[31:28];
        3'd1: hex_val = data_reg[27:24];
        3'd2: hex_val = data_reg[23:20];
        3'd3: hex_val = data_reg[19:16];
        3'd4: hex_val = data_reg[15:12];
        3'd5: hex_val = data_reg[11:8];
        3'd6: hex_val = data_reg[7:4];
        3'd7: hex_val = data_reg[3:0];
        default: hex_val = 4'd0;
    endcase
end

always @(*) begin
    case(hex_val)
        4'd0: seg_pattern = 7'b0111111; // 0
        4'd1: seg_pattern = 7'b0000110; // 1
        4'd2: seg_pattern = 7'b1011011; // 2
        4'd3: seg_pattern = 7'b1001111; // 3
        4'd4: seg_pattern = 7'b1100110; // 4
        4'd5: seg_pattern = 7'b1101101; // 5
        4'd6: seg_pattern = 7'b1111101; // 6
        default: seg_pattern = 7'b0000000;
    endcase
end
Endmodule

```

仿真：

通过运行如下仿真代码：

```
module top_lab5_tb();
```

```

    reg clk;
    reg reset_btn;
    wire [6:0] seg_L;
    wire [6:0] seg_R;
    wire [7:0] an;

```

```
top_lab5 uut (
```

```

        .clk(clk),
        .reset_btn(reset_btn),
        .seg_L(seg_L),
        .seg_R(seg_R),
        .an(an)
    );

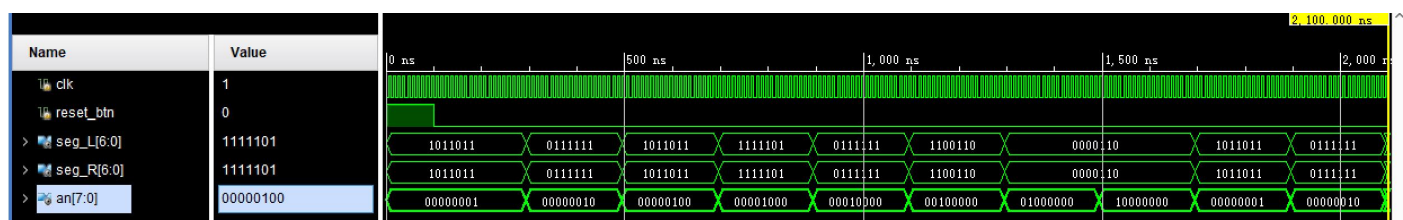
defparam uut.u_div.cnt_scan_max = 17'd19;
defparam uut.u_div.cnt_move_max = 28'd199;
always #5 clk = ~clk;
initial begin
    clk = 0;
    reset_btn = 1;
    #100;

    reset_btn = 0;

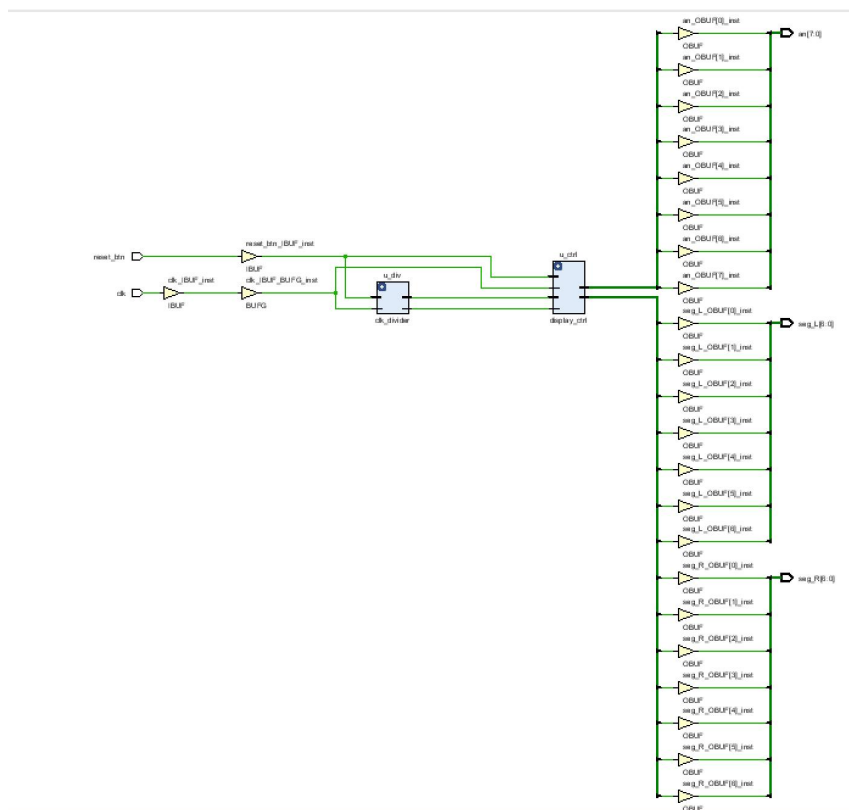
    #2000;
    $stop;
end
endmodule

```

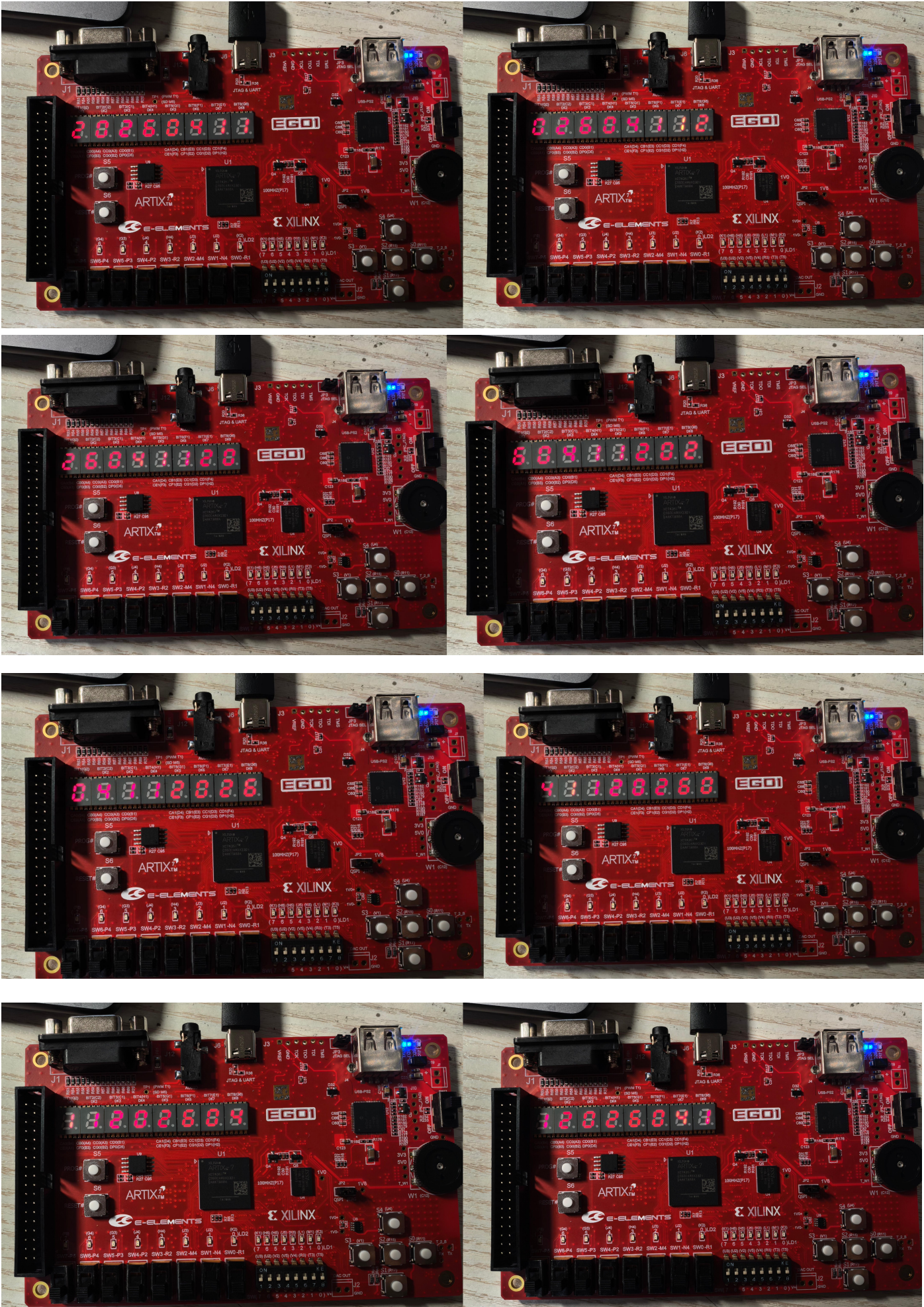
得到仿真输出：



Vivado 中的电路图：



上板运行后成功实现循环滚动：



4 实验结果和分析

实验结果如上，实验顺利完成。我认为实验中的重点在于实现滚动和分频，而滚动实际上是一种移位寄存，将上一数码管的状态传递给下一个数码管，在电路可以通过 D 触发器链式连接实现。而在分频上主要分为扫描频率和移动频率。同时我认为还可以通过将八个数码管个例化的方法让 8 个数码管独立运行同一个状态循环，即将每个数码管定义一个独立的寄存器也可以实现相同的功能。

5. 心得与体会

本次实验让我掌握了数码管的显示原理，也学会了实现循环移位的方法，有效巩固了先前学习的知识。我认为，在这个 lab 中最重要也是最和之前的不同的是对于时序的处理，在解决完这个之后其他的还是比较容易想到。